

Yggdrasil: Reducing Network I/O Tax with (CXL-Based) Distributed Shared Memory

Wenda Tang*
China Telecom eSurfing Cloud
tangwd1@chinatelecom.cn

Ying Han*
China Telecom eSurfing Cloud
hangying1@chinatelecom.cn

Tianxiang Ai
China Telecom eSurfing Cloud
aitianxiang@chinatelecom.cn

Guanghui Li
China Telecom eSurfing Cloud
ligh10@chinatelecom.cn

Bin Yu
China Telecom eSurfing Cloud
yubin1@chinatelecom.cn

Xin Yang
China Telecom eSurfing Cloud
yangxin6@chinatelecom.cn

ABSTRACT

In communication-intensive applications that run on hosts with high-speed network hardware, a common challenge arises from the significant burden placed on the native socket system within the OS. Researchers have devoted considerable effort to optimizing the kernel networking stack and moving the TCP/IP stack to user-space. In this paper, we describe a novel socket replacement solution, *Yggdrasil*, a CXL-based user-space high-performance socket system. *Yggdrasil* is fully compatible with Linux socket, making it a drop-in replacement for existing applications without the need for code modifications. In order to optimize performance, *Yggdrasil* employs CXL-based distributed shared memory (DSM) for inter-host communication whenever it is available. In cases where DSM is not accessible, *Yggdrasil* transparently switches back to Linux socket for communication. A key element in achieving isolation in *Yggdrasil* involves a trusted user-space monitoring daemon responsible for managing control plane operations like connection setup and access control. Within the data plane of *Yggdrasil*, a peer-to-peer model is adopted for communication between processes. To bridge the semantic gap between socket and DSM, we exploit several techniques to ensure compatibility and performance, including (1) transparent dynamic fast/slow data path navigation, (2) decentralized CXL memory management, (3) lock-free queue based QoS-aware dynamic data polling, and (4) semantics-aware memory page migration. By evaluating *Yggdrasil* on both emulated and real CXL hardware, we show that *Yggdrasil* outperforms Linux socket in Memcached throughput by 8.2× and reduces latency by 24~320× in a micro benchmark across different message sizes.

CCS CONCEPTS

• **Software and its engineering** → *Distributed memory*; • **Hardware** → **Emerging architectures**.

*Both authors contributed equally to this work.



This work is licensed under a Creative Commons Attribution International 4.0 License.

ICPP '24, August 12–15, 2024, Gotland, Sweden
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1793-2/24/08
<https://doi.org/10.1145/3673038.3673138>

KEYWORDS

CXL, Socket, Rack-Scale Communication, Disaggregation, Memory Management

ACM Reference Format:

Wenda Tang, Ying Han, Tianxiang Ai, Guanghui Li, Bin Yu, and Xin Yang. 2024. *Yggdrasil: Reducing Network I/O Tax with (CXL-Based) Distributed Shared Memory*. In *The 53rd International Conference on Parallel Processing (ICPP '24)*, August 12–15, 2024, Gotland, Sweden. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3673038.3673138>

1 INTRODUCTION

In modern datacenter environments, achieving high-performance communication between processes, containers, and hosts is essential for optimizing overall system efficiency. This holds particular significance within industries such as e-commerce, where the volume of real-time business transactions is burgeoning, necessitating a heightened emphasis on processing speed [8, 9]. The emerging resource disaggregation architecture further underscores the importance of ultra-low latency and high throughput rack-scale communication within data centers. Traditional socket-based communication, while widely used, often suffers from limitations in terms of latency, throughput, and scalability [20]. To address these challenges, researchers have explored various network-specific approaches, including optimizing the kernel networking stack, moving the TCP/IP stack to user-space, and leveraging ultra-fast networking hardware (e.g., RDMA (Remote Direct Memory Access) enabled SmartNICs [31, 36]) for offloading transport. However, current research on Ethernet-based rack-scale networks has its inherent constraints, including being customized for particular workloads or depending on a centralized controller. These limitations may have adverse effects on overall performance if not addressed effectively. Another area of research concentrates on implementing high-density contemporary computing resources like TPUs [16] and integrating modern storage solutions like Non-Volatile Memory [7] within a rack, which results in the creations of rack-scale computers.

Modern data centers primarily depend on TCP/IP communication protocols for inter-host communication. While TCP/IP has played a critical role in establishing the foundation of the Internet and has proven to be reliable over the years, it does have its limitations. These limitations include increased latency, higher bandwidth usage, and the imposition of a **network I/O tax**. This network I/O tax primarily stems from data processing overhead and memory copying, which can significantly impact network I/O performance.

Concretely, there are three typical overheads associated with the network I/O tax: ① **data encapsulation** involves the computational effort needed to encapsulate data into packets with the necessary headers for transmission, as well as the reverse process of parsing these headers upon data reception. ② **memory copying** overhead arises from the need to transfer data between different memory regions, such as from an application's memory space to the NIC's buffer for sending, and vice versa for received data. ③ **context switching** overhead occurs when the CPU switches between different tasks, such as moving from user-space to kernel-space for network operations, leading to additional processing time and resource overheads. These operations consume valuable CPU resources and introduce latency, which can be significant in networking scenarios, impacting the efficiency and speed of data transfers.

By breaking down the latency of sending data packets over traditional TCP/IP sockets, we find that the aforementioned overheads account for at least 40% of the time expenditure (refer to §2.3 for more details). We believe with the adoption of faster network card devices, this proportion will further increase. However, existing solutions often encounter trade-offs between compatibility, isolation, and performance, and many fail to fully meet the demands of modern datacenter applications, especially in scenarios such as process forking and event polling. These solutions struggle to deliver optimal performance improvements due to issues like multi-thread synchronization, buffer management, and memory copy overheads.

Distributed shared memory (DSM) can be also considered as a potential solution for reducing the impact of network I/O tax. DSM is a shared memory model in distributed systems that enables multiple nodes to share memory space, allowing distributed applications to access shared data as if on a single computer, thereby improving communication efficiency and facilitating data transmission within a data center. However, DSM requires a distributed data synchronization (often by page-level data granularity [15], which frequently lacks application semantics) system between multiple machines over the network to ensure data consistency. To mitigate the expenses associated with network transmission between hosts and ensure data consistency in the context of DSM, we argue that using CXL (Compute Express Link) as the high-speed network technology is an ideal choice to enhance communication efficiency. CXL is an open standard for high-speed interconnectivity, enabling high-bandwidth, low-latency direct memory access between different devices [5]. CXL have brought a range of benefits to data centers and high-performance computing, including load/store access semantics at cache-line granularity [24], improved resource utilization [21], enhanced performance [14], simplified system architecture, support for heterogeneous computing, facilitation of data center modernization, and increased system reliability and flexibility. Through CXL, high-speed, low-latency memory access across multiple hosts can be realized, thereby accelerating data transfer and communication within DSM. It should be also mentioned that the hardware enabled cache coherence shared memory feature of CXL 3.0 further reduces the need for explicit coherence enforcement mechanisms, thereby streamlining communication processes and enhancing overall system performance in DSM systems. As CXL technology continues to develop and mature [32], we can expect it to play an increasingly important role in future computing ecosystems.

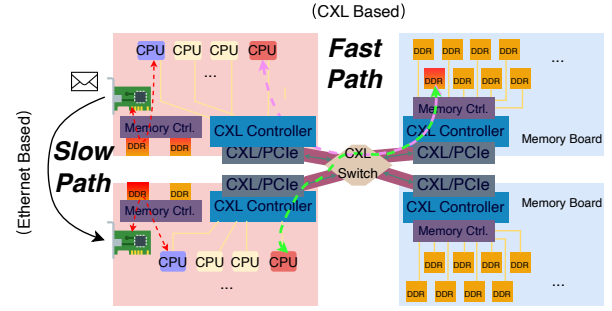


Figure 1: Conceptual representation of fast/slow path communication in Yggdrasil.

Unfortunately, integrating CXL into a network system to enhance communication performance is non-trivial. Challenges such as ensuring compatibility with existing network infrastructures application development, and optimizing software support for CXL integration need to be carefully addressed in order to fully leverage the benefits of CXL for DSM systems. To address these challenges, we present *Yggdrasil*¹, a novel high-performance communication system designed to achieve compatibility, isolation, and exceptional performance simultaneously. Building upon the emerging CXL technology and its inherent hardware controlled DSM paradigms, *Yggdrasil* offers innovative solutions to overcome the limitations associated with conventional socket-based communication methods. *Yggdrasil* transparently intercepts various conventional socket APIs to route data transmission via a CXL-based fast path (when feasible) instead of the traditional slow path typically associated with Ethernet.

Fig. 1 presents a conceptual representation of fast/slow path communication provided by *Yggdrasil*. By leveraging the CXL-based fast path communication, *Yggdrasil* improves the speed and efficiency of data transmission. As far as we know, *Yggdrasil* is the first system leveraging CXL technology for multi-host communication to facilitate transparent, high-performance and efficient communication for socket-oriented rack-scale networks by addressing the challenges of compatibility with existing infrastructures and optimizing software support.

In summary, the contributions of this work include:

- We analysis of existing overheads in traditional socket-based communication. We provide design and implementation of *Yggdrasil*, a high-performance communication system built on CXL (or other hardware distributed cache coherence technologies) that works seamlessly with existing socket APIs.
- We exploit several techniques to ensure compatibility, isolation, and performance for *Yggdrasil*, including transparent dynamic fast/slow data path navigation (§4.1), decentralized CXL memory management (§4.2), lock-free queue based QoS-aware dynamic data polling (§4.3), and semantics-aware memory page migration (§4.4). Their primary goal is to enhance communication efficiency while reducing resource

¹ *Yggdrasil* symbolizes the immense World Tree in Norse mythology, representing a connective network that links the nine realms, much like a rack-scale system interconnecting multiple physical hosts.

overheads like memory copying and wasteful CPU cycles during data transmissions.

- We evaluate *Yggdrasil* on both emulated CXL environment and production-ready CXL hardware. Evaluation demonstrating the performance benefits in both a micro benchmark and a real-world datacenter application, including significant latency reduction and throughput improvement.

The rest of the paper is organized as follows. §2 introduces background and motivates *Yggdrasil*. §3 describes the design concept and provides an overview of *Yggdrasil*. §4 gives implementation details of *Yggdrasil*. We evaluate *Yggdrasil* in §5 and survey related works in §6. §7 concludes this work.

2 BACKGROUND AND MOTIVATION

In this section, we review the concept of distributed shared memory (DSM) and demonstrate the potential benefits and insights of CXL-based DSM for reducing networking I/O tax in rack-scale resource disaggregation by 1) reporting the performance of CXL, the most promising technique for implementing ultra-fast communication; and 2) breaking down the latency within the current kernel stack for message transmission.

2.1 Distributed Shared Memory

Distributed shared memory (DSM) [22] is a computing concept that allows multiple processors or computers to access a shared memory as if it were part of their local memory. DSM provides a continuous virtual memory address space crossing multiple physical machines for applications. DSM systems are built upon the principle of cache coherence, which ensures that multiple processors with caches containing the same memory block have consistent data. The coherence protocol manages the sharing (including true and false sharing) of data blocks among the nodes in a network. When a processor accesses a data block, the system checks if the block is present in the local cache. If not, it is fetched from the remote node where it resides. The whole system must also handle updates to the shared data to ensure that all caches containing the block are updated accordingly. True sharing is processors accessing the same variable, while false sharing is because they each access a different variable on the same data block. There is a trade-off between large coherence blocks to enhance spatial locality and small ones to reduce false sharing [15], while DSM systems usually have large coherence blocks (4KiB), adding to the severity of false sharing. Numerous attempts have been undertaken to minimize false sharing, such as isolating shared data updates to separate physical addresses [23], relocating problematic stack/heap data [17], and adopting relaxed memory consistency models [3, 38].

Insight #1. Although DSM systems facilitate smooth memory sharing and coordination across several processors, they face inherent challenges in managing both true and false sharing efficiently. Hardware enhancements hold significant potential in bridging the performance difference between accessing local and remote memory. With ongoing improvements in network infrastructure and processor architecture, there is a promise of boosting the efficiency and scalability of DSM systems by minimizing the software overhead linked with maintaining data coherence in distributed environments.

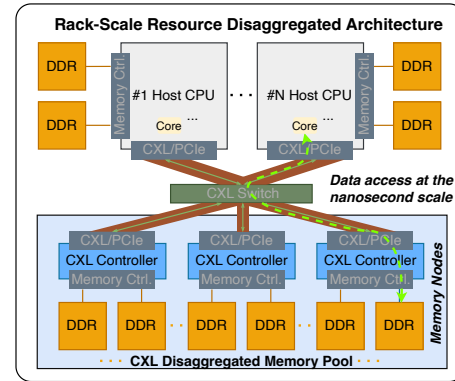


Figure 2: Conceptual representation of rack-scale CXL-based resource disaggregated architecture.

2.2 Compute Express Link

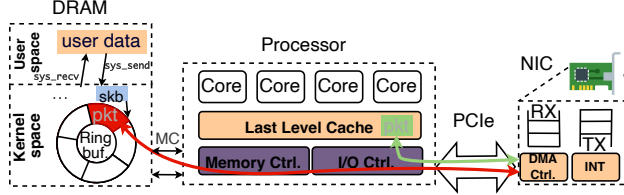
The Compute Express Link (CXL) [5] is a protocol that connects devices using the PCI Express (PCIe) interface as the physical layer. It facilitates linking remote byte-addressable CXL-memory to the host machine’s physical address space, appearing as a CPU-less NUMA node to applications [21]. The CXL Consortium has recently unveiled CXL version 3.0 [33], featuring memory sharing functionalities. Unlike memory pooling in CXL 2.0, the memory sharing capability in version 3.0 enables the CXL switch to map the same remote memory region to multiple physical addresses of host machines concurrently, facilitating simultaneous updates within the same coherency domain (referred to as a CXL-hardware-assisted DSM) [41]. Given that CXL 3.0 has been recently announced and has not yet been put into practice, our analysis relies on a scenario where compute nodes are utilizing an external CXL 2.0 device. The absence of a unified cache coherency domain in the existing hardware setup necessitates additional cache invalidation mechanisms to enable efficient data sharing among multiple machines.

Fig. 2 depicts a conceptual illustration of a rack-scale resource disaggregated architecture based on CXL. It also showcases how DSM could be implemented across multiple compute nodes. With the utilization of retimers and CXL switches, a CPU equipped with CXL support can efficiently access memory located in distant nodes with reduced latency compared to conventional network interface technologies such as RDMA, while reducing memory copying overhead, effectively enabling the process of memory disaggregation. The CXL memory device further enhances the memory disaggregation process by allowing for seamless sharing and utilization of memory resources across different nodes in the architecture. In this setup, every memory request from the host CPU to the CXL memory device requires a CXL protocol conversion (from CPU instructions to one or more CXL flits), which takes a similar or longer amount of time than a DRAM access itself [14], but this architecture grants memory manufacturers unprecedented flexibility in designing and optimizing their memory devices.

To take full advantage of the memory semantics supported by CXL, we configure CXL memory as a CPU-less NUMA node. We use the Intel Memory Latency Checker (MLC) [4] for latency and memory bandwidth measurements on Intel Xeon Platinum 8558P CPU. We show the average and standard deviation (SD) of different

Table 1: Comparisons among local/remote NUMA memory and CXL memory. Demonstrated by Intel MLC [4].

Type	Idle Latency $\pm$ SD	Memory Bandwidth $\pm$ SD				
		All Reads	3:1 Reads-Writes	2:1 Reads-Writes	1:1 Reads-Writes	Stream-triad like
Local NUMA (NUMA 0)	105.79 $\pm$ 0.03 <i>ns</i>	230.18 $\pm$ 3.01 GiB/s	196.86 $\pm$ 0.89 GiB/s	190.45 $\pm$ 2.27 GiB/s	179.13 $\pm$ 3.93 GiB/s	191.17 $\pm$ 0.59 GiB/s
Remote NUMA (NUMA 1)	190.17 $\pm$ 0.04 <i>ns</i>	108.09 $\pm$ 0.67 GiB/s	135.43 $\pm$ 0.69 GiB/s	124.45 $\pm$ 0.89 GiB/s	113.49 $\pm$ 2.59 GiB/s	134.12 $\pm$ 0.63 GiB/s
CXL memory (NUMA 2)	251.82 $\pm$ 0.07 <i>ns</i>	21.57 $\pm$ 0.01 GiB/s	25.77 $\pm$ 0.09 GiB/s	27.61 $\pm$ 0.71 GiB/s	32.61 $\pm$ 0.21 GiB/s	27.59 $\pm$ 0.04 GiB/s

**Figure 3: Conceptual representation of traditional server network architecture.**

load settings with 10 repeated runs. As shown in Table 1, accessing remote CXL memory incurs higher latency compared to accessing memory of another NUMA node on the same machine, though still comparable. This finding aligns with evaluations reported by other researchers [11, 34, 41]. It should be noted that the memory bandwidth of CXL is constrained by PCIe bandwidth. We configure PCIe $\times 8$ for our CXL memory module, and the CXL memory has affinity with NUMA node 0. While CXL memory may have slightly longer data access times than local DRAM, it still provides significant benefits in terms of flexibility for memory manufacturers and efficient communication with the host CPU.

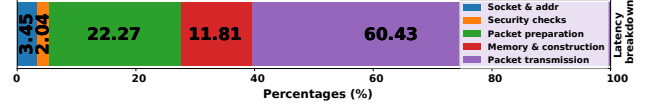
Insight #2. The CXL-based rack-scale architecture for DSM offers a promising path towards efficient memory disaggregation, providing lower latency and increased flexibility for memory sharing compared to traditional technologies. With the upcoming CXL 3.0 standard set to further reduce access times, this approach stands to enhance both performance and resource optimization in distributed computing environments.

2.3 Latency Breakdown of Existing Kernel Stack

The Linux network stack, while successful, does have certain limitations that hinder significant changes and advancements. In fact, the complex multi-provider ecosystem and the reliability of the current architecture are key factors contributing to the stagnation in the evolution of the network stack [1]. Therefore, overcoming these challenges will require collaborative efforts and careful consideration of potential improvements while ensuring the reliability and stability of the network infrastructure.

Fig. 3 depicts the traditional network hardware architecture of a physical server. A Network Interface Card (NIC) is connected to a processor through a PCIe connection. When data is prepared to be sent from a message producer’s perspective, it invokes the `sys_send` system call, which is provided by the modern Linux kernel. Subsequently, the data is transmitted via the NIC to the Ethernet with the assistance of the Linux kernel’s networking abstraction layer.

It should be mentioned that the modern NICs discussed above usually incorporate Data Direct I/O (DDIO) technology [13, 40] to

**Figure 4: Latency breakdown of `inet_sendmsg` function call in Linux kernel.**

optimize memory bandwidth usage during the transmission and reception of network packets. With DDIO, incoming packets are efficiently transferred to a buffer in the processor’s last level cache (LLC) by a Direct Memory Access (DMA) engine, bypassing the need to route the packets all the way to DRAM. Similarly, when transmitting a packet using a NIC with DDIO capability, the packet buffer is allocated within the LLC, and the DMA engine retrieves the packet directly from the LLC, enhancing overall efficiency. This hardware setup ensures efficient data transfer between the NIC and the processor, minimizing latency and optimizing network performance in a server environment. However, the DDIO usually has approximately several MB of cache capacity [40] and often this space is exhausted by a NIC at high RX/TX rates. Moreover, sharing the DDIO space among several network functions can result in a phenomenon known as DMA leakage [27, 28, 37]. The DDIO can cause cache pollution for other applications if there is no upper limit for its LLC share [35, 40]. Because interrupt handling and interrupt moderation can delay the packet processing for several microseconds, ultra-low latency networks are usually deployed in (adaptive) polling mode [10, 18].

We are here to analyze functions involved in the network packet processing trace within the Linux kernel. The trace information is exploited by the Linux `ftrace` system [30]. `Ftrace` is a tool to trace Linux kernel events. The function graph tracer of `ftrace` hijacks the return address of the function in order to insert a trace callback for the function exit. Although this breaks the CPU’s branch prediction and causes a bit more overhead than the function tracer, it still gives us valuable insight into the function execution flow and aids in performance analysis at a granular level within the Linux network architecture. Due to the intricate nature of packet processing within the kernel, we have categorized these functions into the following groups to analyze their processing durations: ① socket operations and address processing (e.g., socket file descriptors lookup, user-kernel space address manipulation); ② security checks; ③ packet preparation (including tasks such as routing determination and socket corking for batch processing); ④ memory allocation and packet construction (like `sk_buff` structure allocation and data transfer between user and kernel spaces. An Ethernet NIC usually employs a circular ring buffer (i.e., descriptor ring) inside the main memory to allow the processor and NIC produce and consume packets at different rates.); and ⑤ packet transmission (involving the operations of network device drivers).

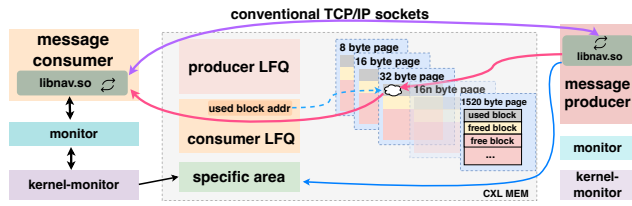


Figure 5: Yggdrasil overview.

Fig. 4 illustrates the breakdown of latency across all the function groups. The breakdown analysis indicates that most of the processing time is taken up during the packet transmission phase, highlighting the significance of efficient network drivers and making use of hardware acceleration. Additionally, the stages of packet preparation and memory allocation offer opportunities for enhancements, such as employing advanced techniques like slab allocation. **Insight #3.** The TCP/IP communication process currently experiences a significant network I/O tax due to the operations involved in handling the protocol stacks. To improve the performance of data transmission, it is crucial for a socket system to minimize this network I/O tax. Additionally, efficient memory management should be implemented to reduce the overhead of packet processing and enhance overall network performance.

3 YGGDRASIL DESIGN OVERVIEW

We design Yggdrasil based on the following guidelines. Firstly, it should not impede the performance of workloads when utilizing CXL memory. Secondly, in cases where CXL memory is inaccessible to the physical host, Yggdrasil should transparently revert to traditional TCP/IP sockets. Lastly, for the practicality of Yggdrasil, it must be capable of functioning effectively across various applications without requiring additional interventions such as code modifications. This section overviews Yggdrasil as shown in Fig. 5.

(1) Transparent fast/slow path navigation. As we not want Yggdrasil to fully replace the conventional TCP/IP sockets, we allow Yggdrasil to attempt to select a faster route using DSM when both communicating peers are within a CXL shared memory domain (CXL domain for short) and seamlessly transition to conventional TCP/IP sockets when the CXL domain is inaccessible. This feature is achieved by transparently intercepting the socket APIs (see libnav.so component in Fig. 5). Initially, the entities will register themselves with the CXL domain and inspect the existence of CXL fast paths for their communication peers. Communication via shared memory is only established if both peers are located within the same CXL domain (e.g., physical hosts located in the same rack and connected to the same CXL memory).

(2) Decentralized CXL memory management. Efficient memory management is critical for ensuring optimal performance and resource utilization within Yggdrasil. Yggdrasil manages rack-scale CXL shared memory, utilizing compare-and-swap (CAS) instructions to control concurrent access to shared memory by multiple physical hosts. It accelerates memory allocation of parallel threads from different physical hosts by pre-allocating isolated memory to reduce conflicts in CXL shared memory access. We designed a set of memory management strategies tailored for CXL memory allocation within Yggdrasil. Through the utilization of CXL shared

memory for data exchange, the network I/O tax is significantly reduced for the communicating peers.

(3) Lock-free queue based QoS-aware dynamic data polling.

When utilizing CXL shared memory for peer-to-peer communication, both the message producer and consumer need to interact efficiently. Yggdrasil adopts a lock-free queue system to facilitate this interaction. Yggdrasil lets a trusted monitor component periodically poll specific data bits in the shared memory to detect the arrival of data (see the monitor component in Fig. 5). To optimize CPU usage from frequent data polling, Yggdrasil piggybacks real-time end-to-end latency (i.e., Quality of Service (QoS)) of peer-to-peer communication in sampled payloads, enabling QoS-aware dynamic data polling.

(4) Semantics-aware memory page migration.

By default, Yggdrasil needs to perform data copying between local memory on the physical host and CXL memory, involving two copies for both communicating peers (i.e., the message producer's data buffer in local DRAM → CXL memory and CXL memory → the message consumer's data buffer in local DRAM). Yggdrasil transparently migrates the memory page of the message producer's data buffer to CXL memory when libnav.so detects a significant amount of redundant memory copy operations. By migrating the data buffer to CXL memory, Yggdrasil simplifies the communication process and improves overall efficiency.

4 DETAILED DESIGN OF YGGDRASIL

§3 provides an overview of the fundamental components of Yggdrasil. However, this representation is a basic model and not directly applicable to a practical deployment system. This section provides the detailed implementation aspects of Yggdrasil.

4.1 Transparent Fast/Slow Path Navigation

To enhance the efficiency of deployment and development, and to reduce the overhead caused by crossing the kernel, we have chosen to implement Yggdrasil in user space rather than kernel space in Linux. The usage of Yggdrasil is quite simple for applications - they only need to load a user-space library called libnav.so by setting the LD_PRELOAD environment variable. By doing so, the application gains the ability to transparently intercept all Linux APIs in the standard C library that are associated with socket operations.

However, it is important to consider the security implications of using libnav.so since it resides within the application's memory address space. Consequently, we cannot fully trust its behavior. For instance, a malicious application could potentially inject arbitrary messages into CXL memory, bypassing the security checks implemented in libnav.so. Yggdrasil maintains a file descriptor (FD) mapping table to differentiate between socket FDs and other kernel FDs in libnav.so. This approach enables libnav.so to handle socket functions in user space, while directing other tasks to the kernel. Therefore, it becomes necessary to have a trusted component external to libnav.so that can enforce access control and allocate necessary resource to support CXL-based communication. We design a monitor daemon on each physical host as the control plane to enforce access control policies, manage address and port resources, and facilitate the establishment of new connections between communicating peers via CXL shared memory. With the

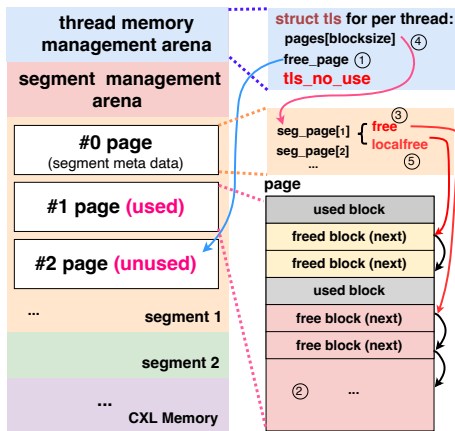


Figure 6: Structure of CXL shared memory pool.

help of the monitor, an application linked to `libnav.so` can inquire whether the communication peer can be connected via the same CXL domain and transparently switch to the fast path. If not, the communication will revert to the slow path based on Linux socket.

4.2 Decentralized CXL Memory Management

For CXL memory management, we designed a decentralized memory management scheme. Every physical host within the same (rack-scale) CXL domain is configured to share one dedicated partition of CXL memory (2GiB in our settings). We use `daxctl` [29] to initialize CXL-attached memory in `devdax` (i.e., device direct access) mode for every CXL-connected physical host. Drawing inspiration from `mimalloc` [19], we allow all the threads from different physical hosts to simultaneously access the same memory partition while having their own memory segments. We designed a set of memory management strategies tailored for CXL memory allocation.

The memory allocation details are illustrated in Fig. 6. Each start-up thread (could from different physical hosts) competes a struct entry for a memory management arena through a CAS operation in the thread memory management arena. This arena is primarily used to store thread-local storage (TLS) structures containing meta information about each thread’s memory allocations (e.g., owner ID is stored in the `tls_no_use` data field). After occupying a dedicated TLS, the thread continues to compete for an exclusive data segment from the segment management arena using CAS. This segment, which is 64MiB in our configuration, is then divided into various data pages, each sized at 64KiB, creating a doubly-linked list.

The `free_page` data field of the TLS points to the first unused available page of all the segments belonging to the thread (refer to ①). When the thread wants to store data (assuming a data size of d) in CXL memory, it should find an available memory block to hold the entire data by querying the `pages` field with the smallest `blocksize` (a multiple of 16) $\geq d$. If not found, it will construct a bunch of memory blocks with `blocksize` in the `free_page` (refer to ②). Then, it updates the `free` field of `seg_page[free_page_idx]` to the first available data block (refer to ③) within the corresponding `pages` data field. The `pages` field will also be updated to reflect the changes (refer to ④). Moreover, it will let `free_page` point to the

next page. Additionally, upon releasing a data block, the `localFree` field will record the corresponding data block location to facilitate the memory reclamation (refer to ⑤).

4.3 LFQ Based QoS-Aware Dynamic Data Polling

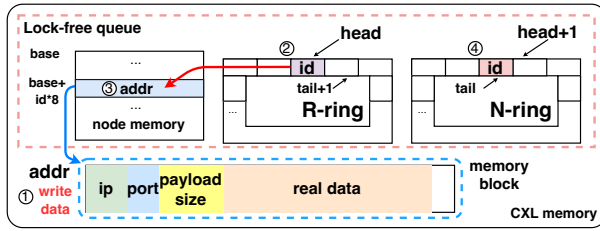
The conventional socket system involves passing through various levels of the TCP/IP protocol stack, before being sent through the NIC. In *Yggdrasil*, the message producer stores data in CXL shared memory, enabling the message consumer to retrieve the data directly from the shared memory. This transformation simplifies the entire process by turning it into a direct memory read/write operation, eliminating the necessity for data transmission over the network. To enhance the efficiency of concurrent read/write operations in a multi-threaded environment, *Yggdrasil* incorporates the communication through a transparent high-performance Lock-Free Queue (LFQ) in CXL memory.

Each message consumer and producer in *Yggdrasil* has its own LFQ for efficient communication. Technically, each LFQ only stores the memory addresses of associated message rather than the message data itself; this decouples the write/read operation of the data from the LFQ management operation. In addition, both LFQ and the data itself are stored in memory blocks in the CXL memory pool. When a message consumer wants to establish its listening address (by using the `bind()` function), it requests its own LFQ entry from the trusted monitor daemon. Once the message consumer thread has obtained its LFQ, it can poll data from the message producers. To facilitate communication, each thread on the message producer side generates a unique port and sends its IP address and port to the monitor in order to acquire its own LFQ. The data from each message producer is written to the consumer's LFQ using data push operations (refer to details in Fig. 7). After the consumer thread reads and processes the data from its LFQ, it could write the response data to the LFQ associated with the producer, where it needs to be returned. On the producer side, the corresponding thread can retrieve the returned data by reading from its own LFQ.

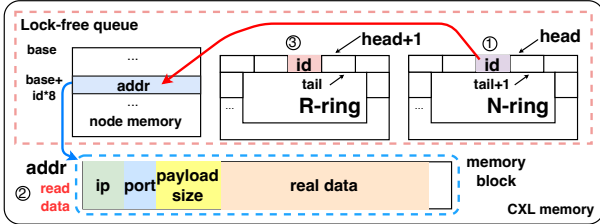
The implementation of LFQ in *Yggdrasil* is referred to as a double-ring lock-free queue, comprising two ring queues and one node memory allocated in CXL shared memory. Fig. 7 provides conceptual representations of data producing and consuming. Specifically, each ring queue has its own head and tail pointer. The tail increments by 1 upon data retrieval from the ring queue, while the head increments by 1 when data is appended into the ring queue. By comparing the numerical values of the head and tail, we can determine if the circular queue has data to read (i.e., tail < head). If R-ring is considered readable, it indicates that the LFQ is writable; if N-ring is considered readable, it indicates that the LFQ is readable.

The process of data push operation (refer to Fig. 7(a)) involves initially requesting a memory segment from CXL memory, writing the relevant data into the memory block (①), getting the next available entry id from the R-ring queue (②), obtaining the corresponding memory address of the entry id location of the node memory, and then writing the memory address (③), and finally appending the entry id to the N-ring (④).

The process of a data pop operation process (refer to Fig. 7(b)) follows similar steps to the data push operation. The message consumer determines the readability of the LFQ by comparing the head and tail of the N-ring. When the LFQ is readable, the entry



(a) Push data to LFQ



(b) Pop data from LFQ

Figure 7: Data transmission between message producer/consumer with push/pop data to LFQ.

id pops up from the N-ring. Then, the data storage address can be retrieved from the corresponding position in the node memory (①). Subsequently, the data is read from the corresponding address (②), and finally, the entry id is written into the R-ring (③). To enable concurrent reading and writing in a multi-threaded environment without relying on distributed locks, CAS operations are also used to validate the read and write capabilities of the circular queue.

The conventional socket system relies on network interrupts to alert the CPU about incoming data packets. However, as far as we know, there is no equivalent mechanism to inform message consumers that the data is accessible in CXL memory. Borrowing ideas from DPDK (Data Plane Development Kit) [6], we use a polling technique to periodically verify the availability of data in LFQ. It is important to note that constant polling is a CPU-intensive strategy. Therefore, we delegate the polling operation from libnav.so to the trusted monitor daemon. Details could be found in Fig. 8. The message producer/consumer could register itself with monitor to delegate the polling operation (①) and yield CPU resources. We developed a kernel module (kernel-monitor) that uses a high-resolution kernel timer (nanosecond scale) to assist the user-space monitor in periodically checking a specific area for incoming new data (②). The specific area holds a magic number to indicate whether the message consumer has some data ready to poll (③). When the kernel-monitor detects the magic number changes, it will wake up the user-space monitor (④). The monitor will then query the information and notify the registered peer to proceed with data reception (⑤ and ⑥).

To further reduce resource overhead of data polling, we designed a QoS-aware dynamic data polling technique. Without loss of generality, we assume that there is some data that needs to be sent from the message producer. The libnav.so in the message producer periodically injects incremental message ID as metadata in each message pushed to LFQ and maintains the timestamps of injected IDs ($\mathcal{N} = \{id : timestamp\}$). Suppose $\mathcal{N} = \{id_i : t_i \dots\}$, when the

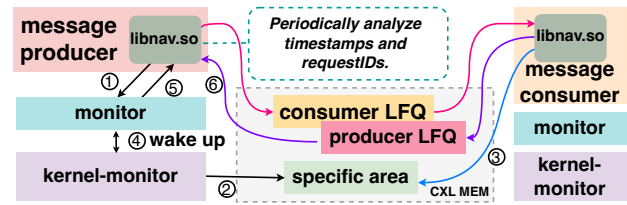


Figure 8: QoS-aware dynamic data polling.

message consumer pops the data from LFQ, the libnav.so in the message consumer will keep the metadata and piggyback it when having some data to reply. Finally, the libnav.so in the message producer will receive the reply and query the corresponding message ID id_i from $\mathcal{N}$, computing the time difference between t_{now} and t_i to represent the end-to-end latency s_i of the message producer and consumer. We consider the series of end-to-end latencies as a streaming data and apply a quantile sketching algorithm DDSketch [25] to summarize the percentile statistics (e.g., P95) of end-to-end latency while requiring minimal memory usage. When the statistics of end-to-end latency violate the target QoS requirement (e.g., P95 latency $> 1ms$), the message producer can inject an emergency message as metadata piggybacked to the message consumer to let it know that the polling interval is not enough to guarantee the target QoS requirement. Alternatively, the polling interval for message consumers could be extended in order to decrease resource usage.

4.4 Semantics-Aware Memory Page Migration

As mentioned before, Yggdrasil requires transferring data between local memory on the physical host and CXL memory by default. This process involves two copies for each communicating peer: copying the data buffer from the message producer's local DRAM to CXL memory and vice versa, from CXL memory to the data buffer of the message consumer in local DRAM.

To improve communication efficiency, Yggdrasil adopts a semantics-aware memory page migration technique to automatically move the memory page of the message producer's data buffer to CXL memory when libnav.so identifies a considerable number of unnecessary memory copy actions. The semantics-aware memory page migration technique plays a vital role in optimizing data transmission efficiency within Yggdrasil. Assuming a scenario where a user-level application sends data by using the sendmsg system call, which involves copying memory from local DRAM to CXL memory. We check whether the associated memory buffer (e.g., 16KiB or larger) has been used before in the libnav.so by keeping track of the virtual address of the memory buffer. If the application repeatedly reuses the same memory buffer for data transmission, it suggests that the memory buffer could be migrated to CXL memory to reduce memory copy overheads.

Technically, before migrating data to CXL memory, we allocate a piece of memory from CXL memory with the same size as the memory buffer and modify the old memory buffer's table entries in its page table to point to the CXL memory. It is important to note that Linux socket depend on the kernel's skb buffer (similar to a message queue middleware) to decouple the user-space memory buffer from the NIC's memory buffer. Therefore, when the sendmsg function (or any other write semantic socket operation) returns, it

indicates that the developer has the ability to modify the user-space memory buffer without impacting the previous sending operation. In *Yggdrasil*, when we decide that some data associated with user-space should be migrated to CXL memory, we use a Copy-On-Write (COW) mechanism to allocate memory replicas for the user's memory writing. After the message consumer pops out the data and frees the associated memory buffer from user space, we reclaim the memory buffer space in the CXL memory to prevent memory leaks and optimize memory utilization within the system. Due to page limits, we omit the details of the implementation of semantics-aware memory page migration in CXL memory.

5 EVALUATION

5.1 Evaluation Methodology

We evaluated *Yggdrasil* on dual-socket servers equipped with Intel Xeon Platinum 8468, where each socket has 48 cores (up to 96 threads with hyperthreading). Both of them are connected via a 1Gbps Ethernet connection. We first utilize remote NUMA to emulate the latency alignment of CXL, which is consistent with prior research efforts [21, 24]. The CXL memory is emulated by reserving memory from remote NUMA via the kernel `mmap` command-line argument. To evaluate latency, we utilize a ping-pong application as a micro benchmark, which allows us to measure the end-to-end time. In order to establish a baseline, we compare the results obtained from this application with those from Linux socket under inter-host communication scenario. By conducting this comparison, we can gain insights into the latency performance and determine any potential improvements.

To further evaluate the performance and effectiveness of *Yggdrasil*, we conducted tests on a real-world application using actual CXL hardware. As CXL 3.0 has recently been announced but not yet implemented in real-world scenarios, our assessment relies on a scenario where two compute nodes utilize an external CXL 2.0 memory device. Both physical servers shared the same CXL memory partition, which was facilitated by the CXL multi-headed device. Table 2 shows the details of the hardware platform. In order to guarantee the persistence of data modifications to CXL memory, we have incorporated `CLFLUSH` instructions in our design. This instruction flushes the cache lines of data to CXL memory, which is essential as we are currently dependent on CXL 2.0 hardware. Nevertheless, in a CXL 3.0-based setup with hardware-distributed cache coherence, this flushing operation may become unnecessary.

Table 2: Platform details.

Model	Intel(R) Xeon(R) Platinum 8468
CPU frequency	2.1 GHz (Max Turbo Frequency 3.80 GHz)
OS	Ubuntu 22.04 with kernel 6.6
Sockets	2
NUMA nodes	2
Local memory	4 * Samsung DDR5 4800 MT/s 32GB
CXL memory capacity	128GB

5.2 Performance Comparison

5.2.1 Emulated CXL memory for a micro benchmark. By comparing the end-to-end latency results of Linux socket in inter-host communication, we observed a **24x to 320x** latency reduction with

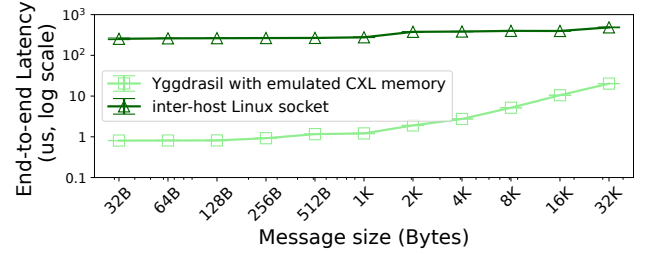


Figure 9: End-to-end latency with various message sizes.

Yggdrasil. This substantial improvement in latency highlights the effectiveness and efficiency of the *Yggdrasil* as compared to the traditional Linux socket.

Fig. 9 depicts the comparison of the end-to-end latency between *Yggdrasil* and the typical Linux socket system across various message sizes ranging from 32B to 32KiB. The y-axis represents the latency on a logarithmic scale, measured in microseconds (us). *Yggdrasil* exhibits significantly lower latency across all message sizes, staying around 10 microseconds even for larger messages. In contrast, the inter-host Linux socket maintains a higher and relatively constant latency around 100 microseconds, with a slight increase as message size grows. As the message size increases, although *Yggdrasil* consistently demonstrates lower latency than Linux socket, the performance gap narrows as the message size expands. While *Yggdrasil* initially shows significantly lower latency compared to the inter-host Linux socket, the rate of increase in latency suggests that its performance advantage diminishes with larger message sizes. This phenomenon can be attributed to the overhead associated with data copying from local memory to CXL memory, which causes the benefits of *Yggdrasil*'s efficient architecture to be diminished. Consequently, although *Yggdrasil* remains superior for small to medium message sizes, its performance gains are less pronounced for larger messages, indicating a scaling limitation when dealing with substantial data transfers.

5.2.2 Effectiveness of semantics-aware memory page migration. We further proactively turn on the semantics-aware memory page migration feature of *Yggdrasil* for all message sizes during transmission. Intuitively, the memory page migration could reduce the data copying overhead between local memory to CXL memory. Fig. 10 shows a comparison between average end-to-end latency and performance improvement for different message sizes when page migration is turned on versus when it is not. By comparing the end-to-end latency results w/ and w/o page migration, we could see an approximate average of 5.6% performance improvement due to page migration to CXL memory. It could also be seen from this figure that for smaller message sizes (32B to 2KiB), the difference in latency between w/ and w/o page migration is minimal. However, as the message size increases, the performance of w/o page migration remains relatively stable, while the latency with page migration appears to increase significantly, especially at 16KiB and 32KiB message sizes. There is a slowdown in the rate of performance improvement between 16KiB and 32KiB message sizes although the percentage of performance improvement continues

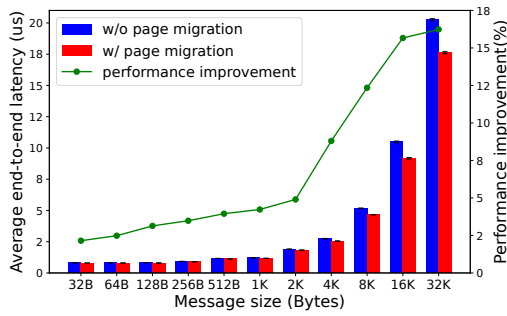


Figure 10: Performance between w/ and w/o page migration of different message sizes.

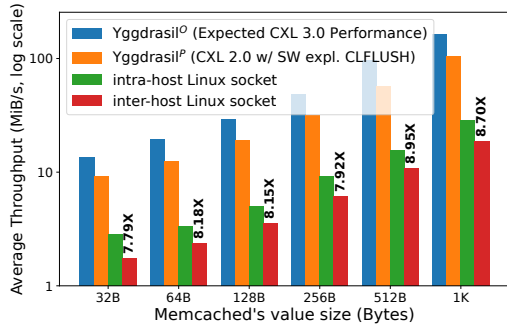


Figure 11: Comparison on data transmission throughput with different value sizes in Memcached.

to rise. This indicates that performance improvements from page migration diminish as message size exceeds 16KiB.

5.2.3 Real CXL memory for a real-world application. We use Memcached [26] (V1.2.7) to demonstrate that *Yggdrasil* can significantly improve the performance of real-world applications without modifying the code. *Memcached* is a typical high-performance system for caching memory objects. It serves as an in-memory key-value store, often acting as a caching intermediary between web servers and databases [2]. We focus on assessing the throughput of Memcached by adopting different communication paths: (1) *Yggdrasil*^{Oracle} (fast path w/ CXL 3.0), (2) *Yggdrasil*^{Practical} (fast path w/ CXL 2.0), (3) intra-host Linux socket (slow path w/o NIC), and (4) inter-host Linux socket (slow path w/ NIC). To generate workloads for our evaluation, we employ libmemcached (V1.0.18) which generates workloads consisting of millions of GET/SET key requests. While maintaining a consistent key size of 57B, we vary the sizes of the values in the GET/SET requests from 32B to 1KiB.

Since CXL 2.0 is not yet capable of creating a single cache coherency domain across different machines, we treat different processes on the same machine in the same manner as those different machines; hence, we can determine the expected performance in CXL 3.0. In contrast, we explicitly add additional cache invalidation mechanisms to share the data between different machines in the current CXL 2.0 device. Fig. 11 provides an analytical comparison of the average throughput achieved by Memcached when handling 1 million requests (set:get=1:9) across varying payload sizes, presented on a logarithmic scale for clarity. We can see there is an average of 35% slowdown between *Yggdrasil*^{Oracle} and *Yggdrasil*^{Practical}, which aligns with evaluations reported by [41]. We can also see

that *Yggdrasil* typically offers the highest throughput across all payload sizes and achieves an average 8.2× the throughput compared to inter-host Linux socket. Although there is an upward trend in throughput as the payload size increases for all of them, Linux socket exhibit consistently lower throughput than *Yggdrasil*. In summary, the evaluation results demonstrate the efficiency of *Yggdrasil* over Linux socket, particularly for larger payloads.

6 RELATED WORK

Ultra-fast network systems. There are several works concerning advanced ultra-fast communications and network systems, such as SocksDirect [20] and NTSocks [12], each leveraging unique technologies to enhance data transfer performance and reduce latency in datacenter environments. SocksDirect is a user-space high-performance socket system that focuses on compatibility and low latency by utilizing RDMA and intra-host shared memory for inter-host and intra-host communication, respectively. NTSocks employs PCIe Non-Transparent Bridge (NTB) technology to create a user-space in-rack interconnect, offering ultra-low latency and compatibility with a fast socket-like abstraction. In comparison, *Yggdrasil* provides a drop-in replacement for socket solution that capitalizes on CXL technology to enable high-speed cache-coherent memory sharing across multiple hosts in a rack, offering a complementary approach to the RDMA and NTB technologies used by SocksDirect and NTSocks.

CXL-based memory disaggregation. Several works have leveraged advanced CXL interconnect technology to enhance resource efficiency via disaggregation [21, 39, 41]. Li *et al.* [21] introduce a CXL-based memory disaggregation approach with memory semantics designed for public clouds. Their method aims to address DRAM inefficiencies caused by memory stranding at the platform level while maintaining optimal performance. They focus on the multiplexing of memory resources across hosts through CXL memory pooling. In comparison, *Yggdrasil* leverages CXL memory sharing to reduce network I/O tax. Wang *et al.* [39] introduce Rcmp, a novel approach that integrates RDMA and CXL technologies to capitalize on their individual advantages. Zhang *et al.* present CXL-SHM [41], a distributed memory management system for resilient and non-blocking handling of partial failures in a CXL based DSM environment.

However, for applications to fully harness the performance enhancements offered by Rcmp and capabilities provided by CXL-SHM, they must be compatible with specific APIs and memory management models, necessitating additional development and testing efforts. In contrast, *Yggdrasil*, besides supporting memory semantics inherent to CXL, emphasizes socket abstraction to offer broader applicability without the need for modification of application. As a result, *Yggdrasil* complements and works independently of these existing works.

7 CONCLUSION

In this paper, we present *Yggdrasil*, a new communication system created to capitalize on CXL's memory semantics capabilities to minimize conventional network I/O tax. It provides a drop-in replacement for Linux socket without requiring code modifications to applications. *Yggdrasil* makes use of CXL-based DSM for inter-host

network communication whenever possible. In cases where this is not feasible, it will revert to Linux socket. Compared to Linux socket, *Yggdrasil* is capable of significantly improving communication performance with the help of CXL-based DSM.

REFERENCES

- [1] Mohammad Alian and Nam Sung Kim. 2019. NetDIMM: Low-latency near-memory network interface architecture. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 699–711.
- [2] Jongsok Choi, Ruolong Lian, Zhi Li, Andrew Canis, and Jason Anderson. 2018. Accelerating memcached on aws cloud fpgas. In *Proceedings of the 9th International Symposium on Highly-Efficient Accelerators and Reconfigurable Technologies*. 1–8.
- [3] Ho-Ren Chuang, Robert Lyerly, Stefan Lankes, and Binoy Ravindran. 2020. Scaling Shared Memory Multiprocessing Applications in Non-cache-coherent Domains. In *Proceedings of the 13th ACM International Systems and Storage Conference*. 13–24.
- [4] Intel Corporation. 2024. *Intel Memory Latency Checker v3.11*. <https://www.intel.com/content/www/us/en/developer/articles/tool/intel-memory-latency-checker.html>
- [5] SM CXL Consortium et al. 2022. Compute express link: The breakthrough CPU-to-device interconnect. Retrieved February 2 (2022), 2023.
- [6] DPK. 2024. *Data Plane Development Kit*. <https://www.dpdk.org/>
- [7] Padmapriya Duraisamy, Wei Xu, Scott Hare, Ravi Rajwar, David Culler, Zhiyi Xu, Jianing Fan, Christopher Kennelly, Bill McCloskey, Danijela Mijailovic, et al. 2023. Towards an adaptable systems architecture for memory tiering at warehouse-scale. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 727–741.
- [8] Peter X Gao, Akshay Narayan, Sagar Karandikar, Joao Carreira, Sangjin Han, Rachit Agarwal, Sylvia Ratnasamy, and Scott Shenker. 2016. Network requirements for resource disaggregation. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 249–264.
- [9] Dan Gibson, Hema Hariharan, Eric Lance, Moray McLaren, Behnam Montazeri, Arjun Singh, Stephen Wang, Hassan MG Wassel, Zhehua Wu, Sunghwan Yoo, et al. 2022. Aquila: A unified, low-latency fabric for datacenter networks. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 1249–1266.
- [10] Hossein Golestani, Amirhossein Mirhosseini, and Thomas F Wenisch. 2019. Software data planes: You can't always spin to win. In *Proceedings of the ACM Symposium on Cloud Computing*. 337–350.
- [11] Minho Ha, Junhee Ryu, Jungmin Choi, Kwangjin Ko, Sunwoong Kim, Sungwoo Hyun, Donguk Moon, Byungil Koh, Hokyoon Lee, Myoungseo Kim, et al. 2023. Dynamic Capacity Service for Improving CXL Pooled Memory Efficiency. *IEEE Micro* 43, 2 (2023), 39–47.
- [12] Yibo Huang, Yukai Huang, Ming Yan, Jiayu Hu, Cunming Liang, Yang Xu, Wenxiong Zou, Yiming Zhang, Rui Zhang, Chunpu Huang, et al. 2022. An ultra-low latency and compatible PCIe interconnect for rack-scale communication. In *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies*. 232–244.
- [13] Ram Huggahalli, Ravi Iyer, and Scott Tetrick. 2005. Direct cache access for high bandwidth network I/O. In *32nd International Symposium on Computer Architecture (ISCA'05)*. IEEE, 50–59.
- [14] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 585–600.
- [15] Xingguo Jia, Jin Zhang, Boshi Yu, Xingyue Qian, Zhengwei Qi, and Haibing Guan. 2022. GiantVM: a novel distributed hypervisor for resource aggregation with DSM-aware optimizations. *ACM Transactions on Architecture and Code Optimization (TACO)* 19, 2 (2022), 1–27.
- [16] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [17] Sang-Hoon Kim, Ho-Ren Chuang, Robert Lyerly, Pierre Olivier, Changwoo Min, and Binoy Ravindran. 2020. DEX: scaling applications beyond machine boundaries. In *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 864–876.
- [18] Steen Larsen, Parthasarathy Sarangam, Ram Huggahalli, and Siddharth Kulkarni. 2009. Architectural breakdown of end-to-end latency in a TCP/IP network. *International journal of parallel programming* 37 (2009), 556–571.
- [19] Daan Leijen, Benjamin Zorn, and Leonardo de Moura. 2019. Mimalloc: Free list sharding in action. In *Programming Languages and Systems: 17th Asian Symposium, APLAS 2019, Nusa Dua, Bali, Indonesia, December 1–4, 2019, Proceedings* 17. Springer, 244–265.
- [20] Bojie Li, Tianyi Cui, Zibo Wang, Wei Bai, and Lintao Zhang. 2019. SocksDirect: Datacenter sockets can be fast and compatible. In *Proceedings of the ACM Special Interest Group on Data Communication*. 90–103.
- [21] Huaicheng Li, Daniel S Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, et al. 2023. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 574–587.
- [22] Kai Li and Paul Hudak. 1986. Memory Coherence in Shared Virtual Memory Systems. In *Proceedings of the Fifth Annual ACM Symposium on Principles of Distributed Computing, Calgary, Alberta, Canada, August 11–13, 1986*, Joseph Y. Halpern (Ed.). ACM, 229–239. <https://doi.org/10.1145/10590.10610>
- [23] Tongping Liu and Emery D Berger. 2011. Sheriff: precise detection and automatic mitigation of false sharing. In *Proceedings of the 2011 ACM international conference on Object oriented programming systems languages and applications*. 3–18.
- [24] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. 2023. TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 742–755. <https://doi.org/10.1145/3582016.3582063>
- [25] Charles Masson, Jee E. Rim, and Homin K. Lee. 2019. DDSketch: a fast and fully-mergeable quantile sketch with relative-error guarantees. *Proc. VLDB Endow.* 12, 12 (aug 2019), 2195–2205. <https://doi.org/10.14778/3352063.3352135>
- [26] memcached. 2024. *memcached*. <https://memcached.org/>
- [27] Rolf Neugebauer, Gianni Antichi, José Fernando Zazo, Yury Audzevich, Sergio López-Buedo, and Andrew W Moore. 2018. Understanding PCIe performance for end host networking. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 327–341.
- [28] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. 2019. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 361–378.
- [29] pmem.io. 2024. *Ndctl and daxctl*. <https://github.com/pmem/ndctl>
- [30] Steven Rostedt. 2024. *ftrace - Function Tracer*. <https://www.kernel.org/doc/html/latest/trace/ftrace.html>
- [31] André Ryser, Alberto Lerner, Alex Forencich, and Philippe Cudré-Mauroux. 2022. D-RDMA: Bringing Zero-Copy RDMA to Database Systems. In *CIDR*.
- [32] Debendra Das Sharma. 2023. Compute Express Link™(CXL™): An Open Interconnect for Cloud Infrastructure. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–4.
- [33] D Das Sharma and Ishwar Agarwal. 2022. Compute Express Link 3.0. *white paper, CXL Consortium* (2022).
- [34] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, et al. 2023. Demystifying cxl memory with genuine cxl-ready systems and devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 105–121.
- [35] Dan Tang, Yungang Bao, Weiwei Hu, and Mingyu Chen. 2010. DMA cache: Using on-chip storage to architecturally separate I/O data from CPU data for improving I/O performance. In *HPCA-16 2010 The Sixteenth International Symposium on High-Performance Computer Architecture*. IEEE, 1–12.
- [36] Konstantin Taranov, Steve Byan, Virendra Marathe, and Torsten Hoeftler. 2022. Kafkadirect: Zero-copy data access for apache kafka over rdma networks. In *Proceedings of the 2022 International Conference on Management of Data*. 2191–2204.
- [37] Amin Tootoonchian, Aurojit Panda, Chang Lan, Melvin Walls, Katerina Argyraki, Sylvia Ratnasamy, and Scott Shenker. 2018. ResQ: Enabling SLOs in Network Function Virtualization. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. 283–297.
- [38] Keval Vora, Sai Charan Koduru, and Rajiv Gupta. 2014. ASPIRE: Exploiting asynchronous parallelism in iterative algorithms using a relaxed consistency based DSM. *ACM SIGPLAN Notices* 49, 10 (2014), 861–878.
- [39] Zhonghua Wang, Yixing Guo, Kai Lu, Jiguang Wan, Daohui Wang, Ting Yao, and Huatao Wu. 2024. Rcmp: Reconstructing RDMA-Based Memory Disaggregation via CXL. *ACM Transactions on Architecture and Code Optimization* 21, 1 (2024), 1–26.
- [40] Yifan Yuan, Mohammad Alian, Yipeng Wang, Ren Wang, Ilia Kurakin, Charlie Tai, and Nam Sung Kim. 2021. Don't forget the I/O when allocating your LLC. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 112–125.
- [41] Mingxing Zhang, Teng Ma, Jinqi Hua, Zheng Liu, Kang Chen, Ning Ding, Fan Du, Jinlei Jiang, Tao Ma, and Yongwei Wu. 2023. Partial Failure Resilient Memory Management System for (CXL-based) Distributed Shared Memory. In *Proceedings of the 29th Symposium on Operating Systems Principles (Koblenz, Germany) (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 658–674.